

Emporika : Chatbot IA pour PrestaShop

Le chatbot IA [Emporika](#) pour PrestaShop 8.1+ et 9 est un vendeur en ligne qui conclut des ventes dans votre boutique : le client décrit ce qu'il cherche ou téléverse la photo d'un article qui lui plaît, le chatbot trouve les produits correspondants dans votre catalogue, gère les objections comme « trop cher » en proposant des alternatives plutôt qu'une remise, répond aux questions à partir de vos pages CMS et l'accompagne jusqu'au panier et au paiement, en 65+ langues. Ce module synchronise votre catalogue produits et vos pages CMS avec Emporika, insère le widget de chat sur votre vitrine et expose les endpoints qui gèrent le panier et le suivi de commande dans le chat.

[image]

- **Présentation de l'intégration** : emporika.com/fr/integrations/prestashop/
- **Documentation complète** : emporika.com/fr/docs/prestashop/ (référence du format webhook, exemples de hooks, dépannage)
- **Fonctionnalités** : emporika.com/fr/features/ · **FAQ** : emporika.com/fr/faq/ · **Tarifs** : emporika.com/fr/pricing/
- **Démo en ligne** : demo.emporika.com et une [vidéo de 30 secondes](#). Cette démo vend de l'électronique, et le comportement est le même sur n'importe quel catalogue.

Prérequis

- PrestaShop 8.1+ ou 9.x
- PHP 7.4+ sur PrestaShop 8.1, PHP 8.1+ sur PrestaShop 9
- Un [compte Emporika](#). Aucune carte bancaire demandée, et 25 \$ de crédit (environ 100 conversations) appliqué automatiquement à

l'ouverture du compte

Installation

1. Téléchargez le module sur [PrestaShop Addons](#).
2. Dans votre back office PrestaShop, allez dans **Modules > Gestionnaire de modules > Téléverser un module** et envoyez `emporiqa.zip`.
3. Cliquez sur **Configurer** sur le module Emporiqa.
4. Cliquez sur **Se connecter à Emporiqa**. Un nouvel onglet s'ouvre sur `emporiqa.com`. Créez un compte gratuit (sans carte, 25 \$ de crédit à l'inscription) ou connectez-vous si vous en avez déjà un, puis choisissez la boutique à connecter (ou créez-en une nouvelle). Le module est connecté à votre retour.
5. Sur l'onglet **Sync**, cliquez sur **Envoyer mon catalogue**. Produits, pages et déclinaisons remontent ; le widget apparaît sur votre vitrine dès que le premier produit arrive.

Site en HTTP, ou vous préférez coller les identifiants vous-même ?

Dépliez **Modifier les identifiants manuellement** sur la page Configurer. Collez un **Store ID** et un **Connection Secret** repris de votre tableau de bord Emporiqa, sous **Paramètres → Intégration Boutique**. Les deux chemins mènent au même résultat.

Pour le suivi de commande, copiez l'**URL de suivi de commande** affichée sur la page Configurer et collez-la dans votre tableau de bord Emporiqa, sous **Intégration Boutique → Suivi de Commande** (dans la plupart des cas, la connexion en un clic déduit aussi cette URL toute seule).

Configuration

Tous les paramètres sont gérés depuis la page de configuration du module (**Modules > Emporiqa > Configurer**) :

Paramètres de connexion

La méthode recommandée est **Se connecter à Emporika** (échange signé en un clic, aucun identifiant à coller). Sur les sites en HTTP, ou pour une configuration manuelle, dépliez **Modifier les identifiants manuellement** :

Paramètre	Description	Par défaut
Store ID	Votre identifiant de boutique Emporika (rempli automatiquement par la connexion en un clic)	(aucun)
Connexion Secret	Secret de signature HMAC-SHA256 (rempli automatiquement par la connexion en un clic)	(aucun)
URL de suivi de commande	Endpoint en lecture seule à coller dans votre tableau de bord Emporika	auto-généré

Avancé

Paramètre	Description	Par défaut
Sync Produits	Activer la sync produits en temps réel	Activé
Sync Pages	Activer la sync des pages CMS en temps réel	Activé
Langues activées	Langues incluses dans les payloads de sync	Toutes les langues actives de la boutique
URL Webhook	Endpoint webhook Emporika	<code>https://emporika.com/webhooks/sync/</code>
Taille de lot	Produits/pages par requête webhook lors de la sync en masse	25

Le suivi de commande (avec vérification de l'email du client) et les opérations de panier dans le chat sont toujours activés. Aucune configuration nécessaire.

Mention de l'IA

Le message d'accueil par défaut indique au client qu'il s'adresse à l'assistant IA de la boutique, dans chacune des langues que le chat parle. Un message d'accueil personnalisé doit conserver cette mention ; s'il l'omet, il est refusé à l'enregistrement. La section 8.6 des [conditions d'utilisation d'Emporika](#) considère la suppression de cette mention, y compris par du CSS ou du code personnalisé, comme un manquement au contrat.

Garder votre catalogue à jour

Le module envoie automatiquement à Emporika les modifications de produits, de pages et de commandes, au fil de l'eau, via les hooks PrestaShop. Celles qui ne touchent qu'un produit, promotion programmée (SpecificPrice), changement d'image ou de déclinaison, déclenchent d'elles-mêmes un nouvel envoi du produit concerné ; un simple changement de stock ou une mise en rupture envoie une mise à jour compacte, limitée à la disponibilité, sans reconstruire le produit entier.

Certains changements touchent l'ensemble du catalogue (renommage de catégories ou de marques, rafraîchissement des taux de change, modification de taux de TVA ou de groupes de règles de TVA, modification de règles panier, nouvelle langue activée). Relancer depuis ces hooks une synchronisation produit par produit, de façon synchrone, bloquerait la requête admin ; le module se contente donc d'enregistrer un avertissement dans **Paramètres avancés** → **Journaux** et laisse le rafraîchissement du catalogue à un lancement manuel.

Relancez une synchronisation complète depuis l'onglet **Sync** quand :

- Vous voyez l'un des avertissements de « changement à l'échelle du catalogue » dans le journal PrestaShop
- Vous ajoutez une nouvelle boutique en mode multi-boutique (les produits existants ne porteront pas les données de cette boutique

- tant qu'une autre opération ne les aura pas modifiés)
- Vous importez des produits en masse depuis un fichier CSV (PrestaShop contourne parfois les hooks d'enregistrement standards lors des imports en masse)
- Un script personnalisé, une migration ou un autre module écrit des données catalogue directement en base
- Emporiqa a été injoignable pendant une période prolongée (panne réseau, maintenance planifiée, identifiants expirés)

Par sécurité, lancez une synchronisation complète une fois par semaine, pour rattraper une éventuelle dérive due à un envoi en arrière-plan qui aurait échoué.

Champs du payload produit

Au-delà des champs présentés dans la [référence du payload webhook](#), le payload complet du produit et de ses déclinaisons porte ces champs natifs PrestaShop, de merchandising et de prix :

- `condition` : chaîne ou null ; la `condition` du produit PrestaShop ("new", "used" ou "refurbished").
- `is_virtual` : booléen ; vrai pour les produits dématérialisés sans expédition.
- `available_for_order` : booléen ; faux pour les produits en mode catalogue, en affichage seul. L'assistant les décrit toujours, mais ne les ajoute pas au panier.
- `max_order_quantities` : dictionnaire par canal (`{canal: int | null}`) de la quantité maximale autorisée par commande. PrestaShop n'ayant pas de maximum natif par commande, ce champ vaut pour l'instant toujours `null` (aucune limite). Il est là pour la parité de contrat entre plateformes, afin qu'une source personnalisée puisse le renseigner plus tard.
- `tier_prices` : liste par devise des remises sur quantité et des tarifs dégressifs (`[{min_quantity, price}]`), présente sur une entrée de prix uniquement si le produit ou la déclinaison a des remises sur quantité configurées dans PrestaShop. Chaque palier reprend le prix unitaire affiché au visiteur non connecté à ce seuil. Les paliers réservés à un groupe, à un client ou à un pays (B2B)

sont volontairement exclus.

Ces champs font partie du payload complet produit et déclinaison, pas de l'événement léger `product.availability`, qui ne porte que le numéro d'identification, le SKU, les statuts de disponibilité par canal et les quantités en stock.

Structure du module

```
emporiqa/
├── emporiqa.php                # Classe principale du module
├── config.xml                  # Métadonnées du module
├── logo.png                    # Icône du module
├── classes/
│   ├── EmporiqaCartHandler.php    # Opérations de panier
│   ├── EmporiqaChannelResolver.php # Mappage multi-boutique
│   ├── EmporiqaLanguageHelper.php # Utilitaires de langage
│   ├── EmporiqaOrderFormatter.php  # Formatage du payload
│   ├── EmporiqaPageFormatter.php   # Formatage du payload
│   ├── EmporiqaProductFormatter.php # Formatage du payload
│   ├── EmporiqaSignatureHelper.php # Signature et vérification
│   ├── EmporiqaSyncService.php     # Orchestration de sync
│   └── EmporiqaWebhookClient.php    # Client HTTP pour webhooks
├── controllers/
│   ├── admin/
│   │   ├── AdminEmporiqaController.php    # Redirect
│   │   └── AdminEmporiqaConnectController.php # Handshake
│   └── front/
│       ├── cartapi.php                    # Endpoint API panier
│       └── ordertracking.php              # Endpoint suivi de commande
├── views/
│   ├── css/admin.css                     # Styles de configuration
│   ├── img/                              # Images du module
│   ├── js/
│   │   ├── admin-sync.js                 # UI de sync en masse
│   │   └── front-cart-handler.js         # Intégration panier
│   └── templates/
```

```

|      |— admin/configure.tpl      # Template de la pa
|      |— admin/sync_tab.tpl    # Template de l'ong
|      |— hook/header.tpl      # Embed du widget (
|— translations/              # Catalogues de tra
|— upgrade/                   # Scripts de mise à

```

Hooks PrestaShop enregistrés

Hook	Fonction
displayHeader	Intègre le widget sur la boutique
actionProductSave	Synchronise le produit lors de la création/modification
actionProductDelete	Envoie l'événement de suppression pour le produit et ses variantes
actionObjectCombination{Add, Update, Delete} After	Synchronise le produit parent quand les déclinaisons changent
actionObjectCms{Add, Update, Delete} After	Synchronise les CMS à la création/modification/suppression
actionValidateOrder	Capture l'ID de la commande et envoie l'événement order.completed
actionOrderStatusPostUpdate	Envoie order.completed pour les captures de paiement tardives
actionUpdateQuantity	Émet un événement product.available quand le stock change (sans reconstruire la vue complète du produit)

<code>actionProductOutOfStock</code>	Émet un événement <code>product.outOfStock</code> lors des transitions de stock au-dessous du seuil de stock
<code>actionObjectSpecificPrice{Add, Update, Delete}After</code>	Re-synchronise les prix concernés lors des modifications programmées, regroupées par groupe et regroupées par quantité (tarifs de promotion)
<code>actionObjectImage{Add, Update, Delete}After</code>	Re-synchronise les images concernées quand les images changent
<code>actionObjectCategory{Update, Delete}After</code>	Enregistre un avertissement pour marchand lance la synchronisation (impact à l'échelle catalogue)
<code>actionObjectManufacturer{Update, Delete}After</code>	Enregistre un avertissement pour marchand lance la synchronisation (impact à l'échelle catalogue)
<code>actionObjectCartRule{Add, Update, Delete}After</code>	Enregistre un avertissement pour marchand lance la synchronisation (impact à l'échelle catalogue)
<code>actionObjectCurrencyUpdateAfter</code>	Enregistre un avertissement pour marchand lance la synchronisation (impact prix à l'échelle catalogue)

<code>actionObjectTaxUpdateAfter / actionObjectTaxRulesGroupUpdateAfter</code>	Enregistre un avertissement p marchand lance synchronisation (impact prix à l'é catalogue)
<code>actionObjectLanguageAddAfter</code>	Enregistre un avertissement p marchand lance synchronisation (nouvelle locale compléter)

Hooks d'extensibilité

Les développeurs peuvent se brancher sur le pipeline de sync pour personnaliser les payloads ou annuler des syncs :

Hook	Fonction	Paramètres
<code>actionEmporiquaFormatProduct</code>	Modifier le payload produit/ variation avant envoi	<code>&\$data, \$product, \$event_ty</code>
<code>actionEmporiquaFormatPage</code>	Modifier le payload page avant envoi	<code>&\$data, \$p, \$event_ty</code>
<code>actionEmporiquaFormatOrder</code>	Modifier le payload de suivi de commande	<code>&\$data, \$o</code>
<code>actionEmporiquaShouldSyncProduct</code>	Annuler conditionnellement une sync produit	<code>\$product, \$event_ty, &\$should_</code>

actionEmporiqaShouldSyncPage	Annuler conditionnellement une sync page	\$page, \$event_type & \$should_
actionEmporiqaWidgetParams	Modifier les paramètres d'intégration du widget de chat	& \$params
actionEmporiqaOrderTracking	Modifier la réponse de suivi de commande	& \$data, \$o

Tarifs

Le module est payant sur PrestaShop Addons et gratuit sur [GitHub](#). Le service Emporiqa, lui, est facturé à l'usage : 0 \$/mois de base + 0,25 \$/conversation, avec 25 \$ de crédit à l'ouverture du compte et aucune carte bancaire demandée à l'inscription. Tarifs complets sur emporiqa.com/fr/pricing/.

Support

Écrivez à support@emporiqa.com.

Licence

[Academic Free License 3.0 (AFL-3.0)](<https://opensource.org/licenses/AFL-3.0>)